Volume 04 | Calendar Rules, Cleaning, and Synthetic Data

Understand calendar semantics, generated records, and data provenance.

1. Cleaning and station roster

`pipeline/01_audit_and_clean_data.py` processes the raw station seed file into cleaned station metadata and related operational inputs. Runtime reads `data/cleaned/stations_cleaned.csv` (routes has legacy-path fallback to `csvFiles/DELHI_METRO_DATA.csv`). Current cleaned table contains 243 station records. This is a project roster count, not a verified unique physical count of all official Delhi Metro stations.

2. Generation dimensions

Step 02 starts Monday 2025-01-06, spans 104 weeks/728 days, and emits 19 hour slots from 05:00 through 23:00 for each station. Current dimensions: 243 x 728 x 19 = 3,361,176 rows. Output is `data/synthetic/dmrc_calibrated_footfall.csv`; generator also writes typology mapping and line distribution model.

3. How synthetic labels are composed

Station weights incorporate line throughput assumptions, interchange multiplier and named hub factors. Date scenarios include day-of-week multipliers, hourly demand shape, seasonal multipliers, holidays/festival factors, station typology and seeded random variation. Project-authored network daily-entry benchmarks calibrate overall scale. Those benchmark values are generator assumptions; do not say they were observed counts.

4. Calendar module semantics

`get_calendar_context(date)` checks occasion records, weekend and holiday flag. It returns weekday, workday, public holiday, day status, occasion, category, numeric holiday type and source text. Weekend wins day-status labeling; public holiday is distinct; future festival dates can be marked as listed while office closure remains unconfirmed. Fixed annual holiday dates are added for future years. The coverage text explicitly notes that only major occasions are enumerated; weekday is calculated for any date.

Caution: “holiday type” is a model feature and event class, not a measured attendance effect. Future dates may have a known festival day but uncertain government closure. Do not infer passenger response directly from the holiday label.

5. Operating hours versus sample slots

`operating_hours.py`: start 05:00 inclusive; end 23:30 exclusive. Forecast model slots are integer hours 05-23. At 23:15 service is considered open but hourly profile uses the 23 slot. At 23:30 closed. At 04:59 closed, at 05:00 open. Boundary tests should cover these exact times.

6. Pandas/NumPy use

Pandas reads CSVs, merges station attributes, groups prior history and stores generated rows. NumPy vectorizes trig encodings, random number generation and numerical transforms. The giant generated CSV is for offline model training; runtime mainly reads compact metadata/model and small station/operations data. Deployment must package the model artifacts and compact runtime inputs, not assume training CSV will be present unless retraining is part of the deployment build.

7. Provenance and caveats

Data categories: cleaned station seed, hand-authored topology/config, synthetic target labels, trained model artifacts. There is no observed AFC/tap-in/out ridership dataset in this project snapshot. Keep provenance in the UI/API, and never blend future real data with synthetic labels without an explicit target/source flag.

Exercises

Calculate row count from dimensions. Compare 23:15 with 23:30. Explain distinction between weekday, holiday category, and service-open. Name which inputs are model training versus runtime. Design a minimal row provenance record for future observed counts.